Representação de Árvores

Estruturas de Dados



UnB | CIC

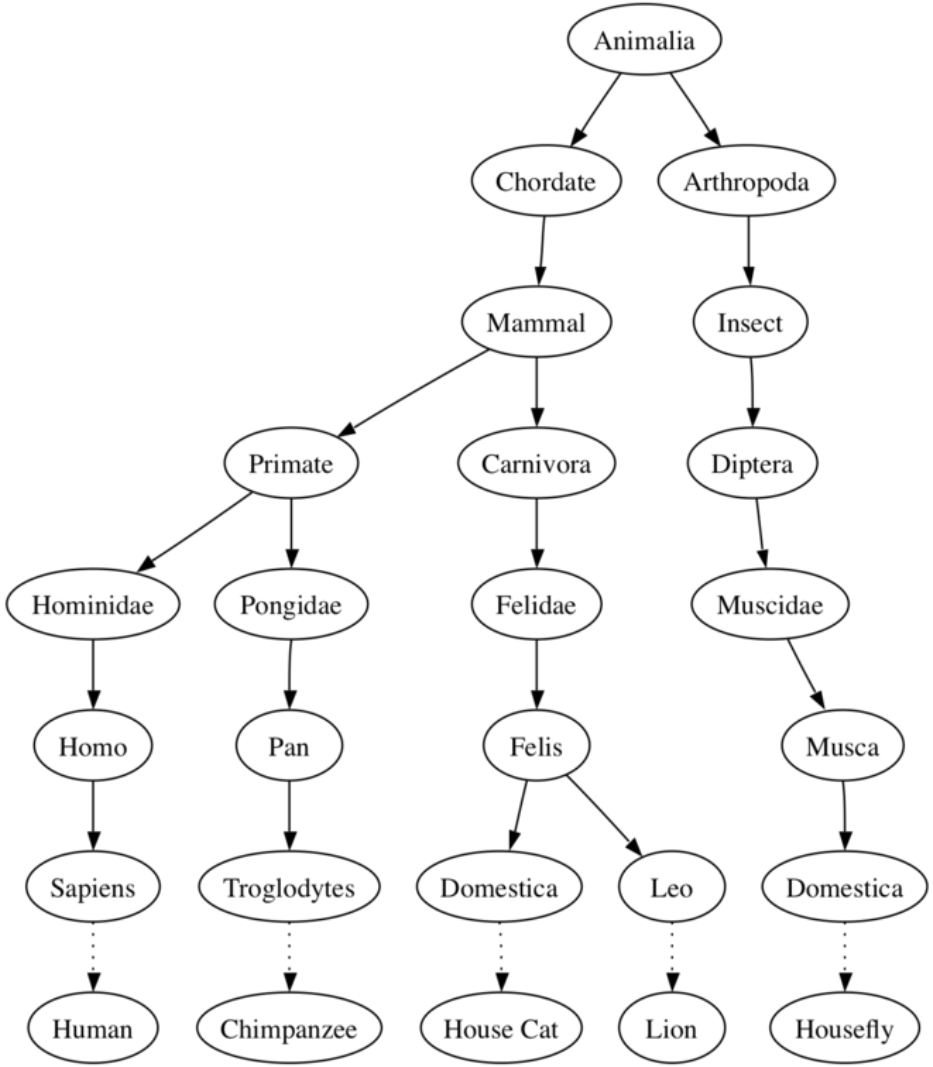
Definição

Formalmente, definimos uma árvore *T* como um conjunto finito de zero ou mais nodos tal que:

- se o número de nodos = 0, temos uma árvore vazia, ou
- se o número de nodos > 0
  - existe um nó especialmente denominado **raiz** de *T*
  - os nós restantes formam $m \geq 0$ conjuntos disjuntos $p_1, p_2, \ldots, p_m$ cada um desses conjuntos é uma árvore em si, chamada subárvore da raiz de *T*, ou simplesmente subárvore.

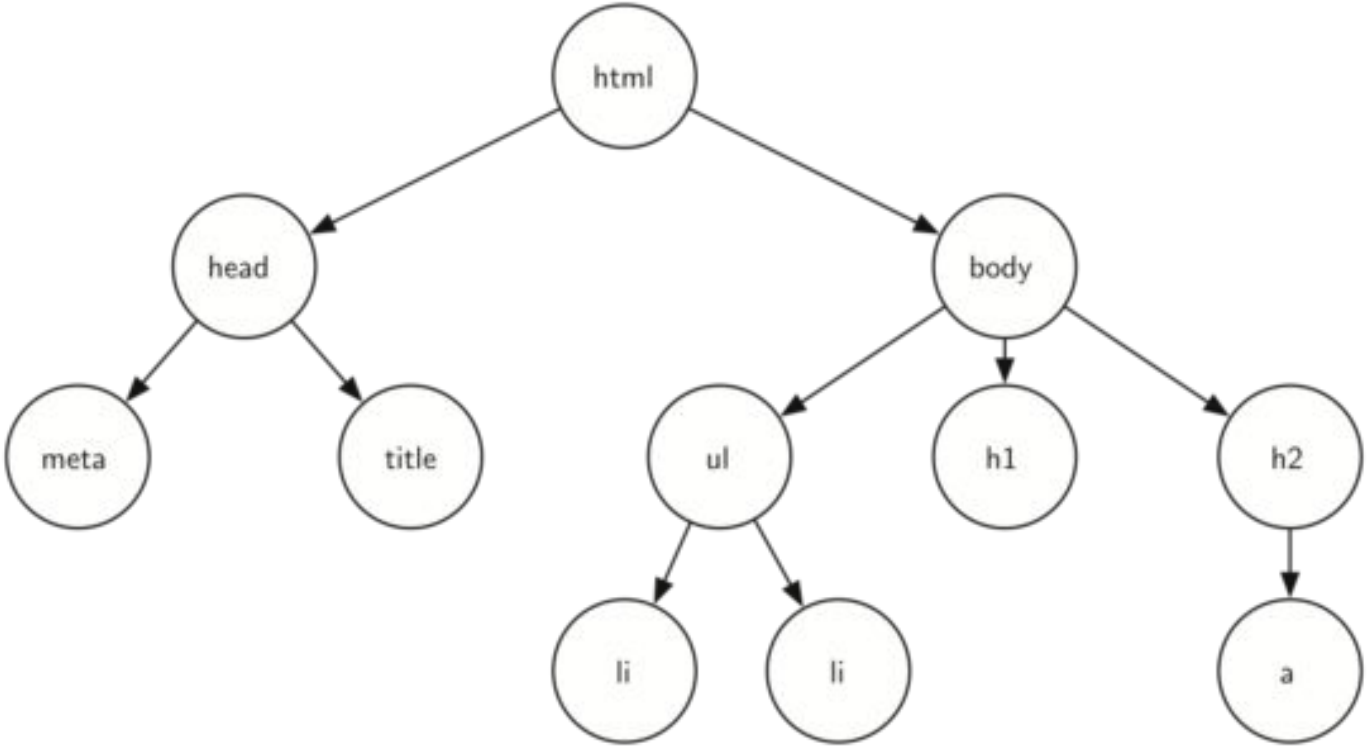
Exemplos

Taxonomia biológica



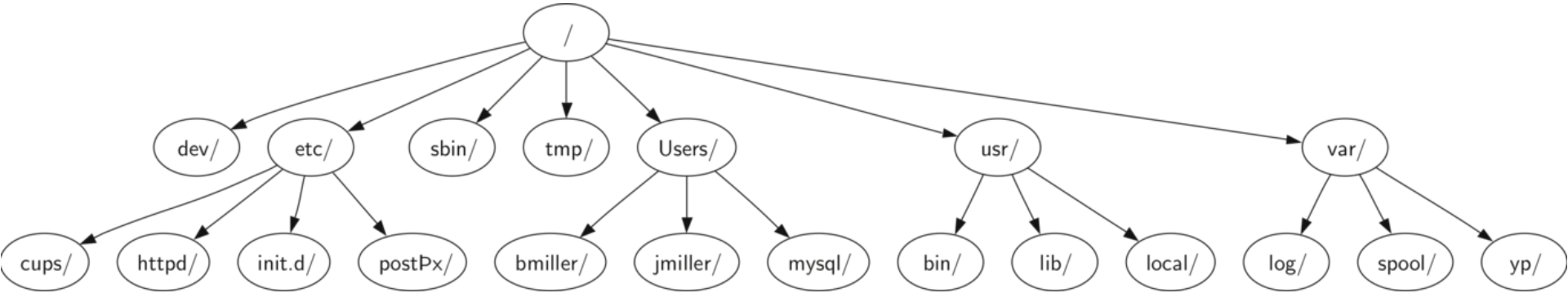
Exemplos

Página HTML



Exemplos

Sistema de arquivos UNIX



Propriedades

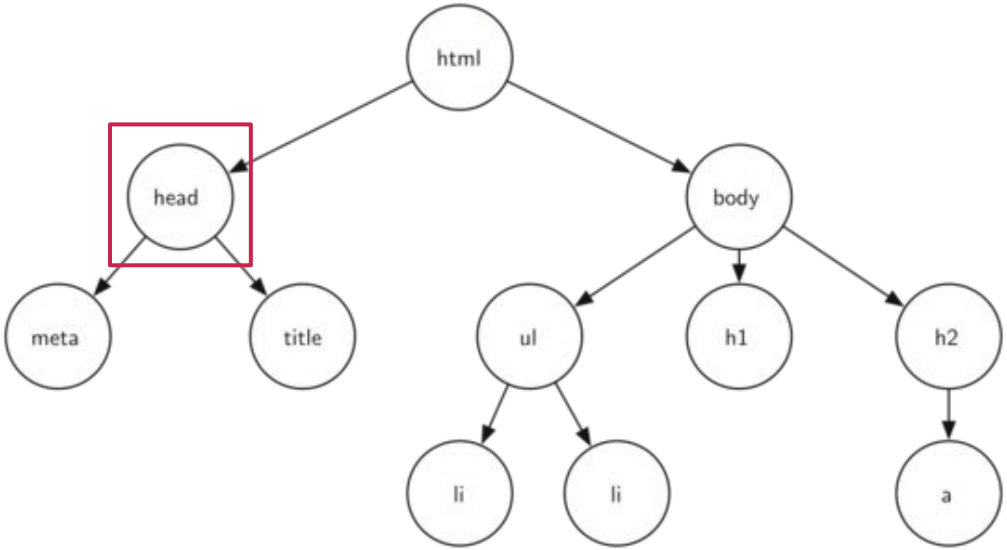
1. Árvores são *hierarquias* - camadas com informações mais gerais perto da parte superior e informações mais específicas na parte inferior.
2. Todos os filhos de um nó são independentes dos filhos de outro nó.
3. Cada nó folha é único.

Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura

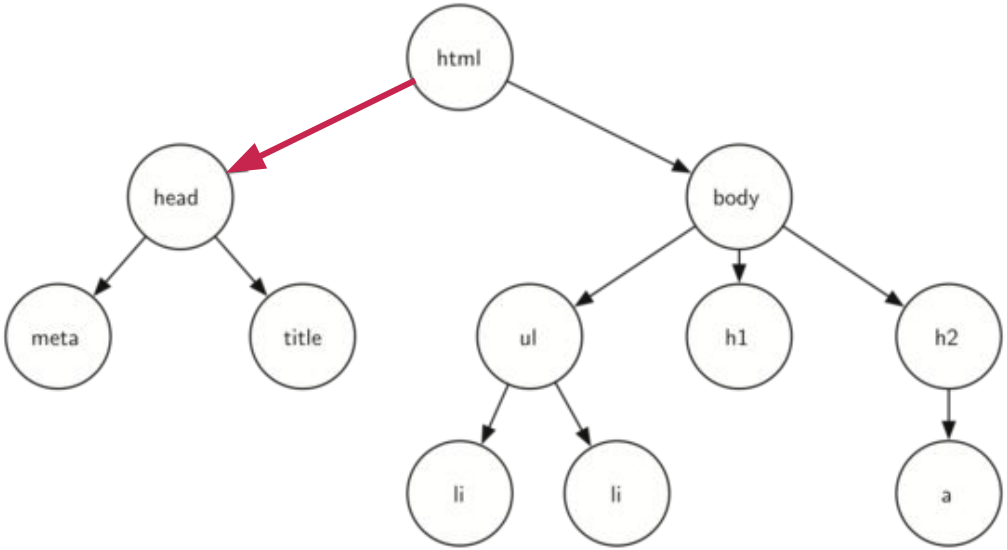
Vocabulário

- **Nó**
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



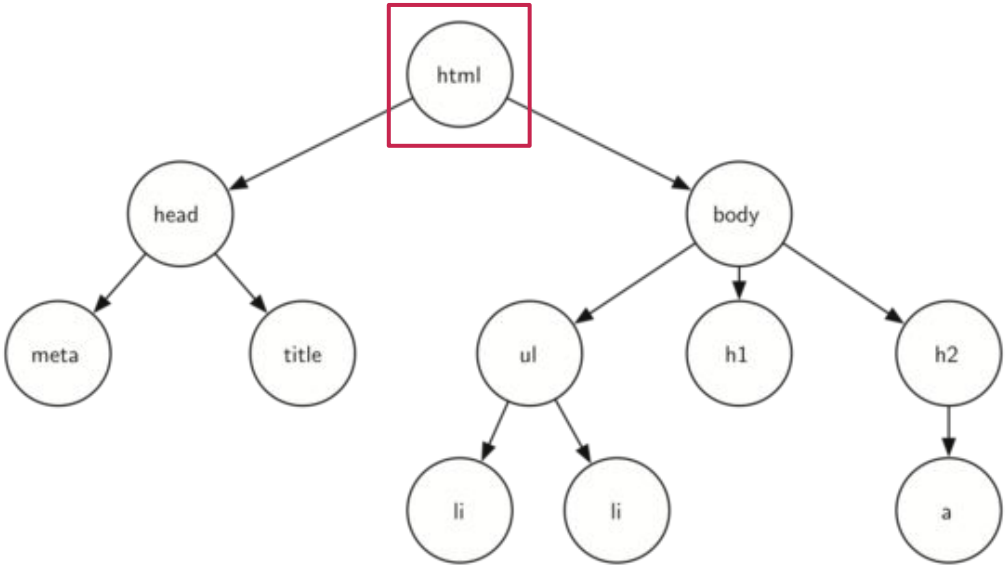
Vocabulário

- Nó
- **Aresta**
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



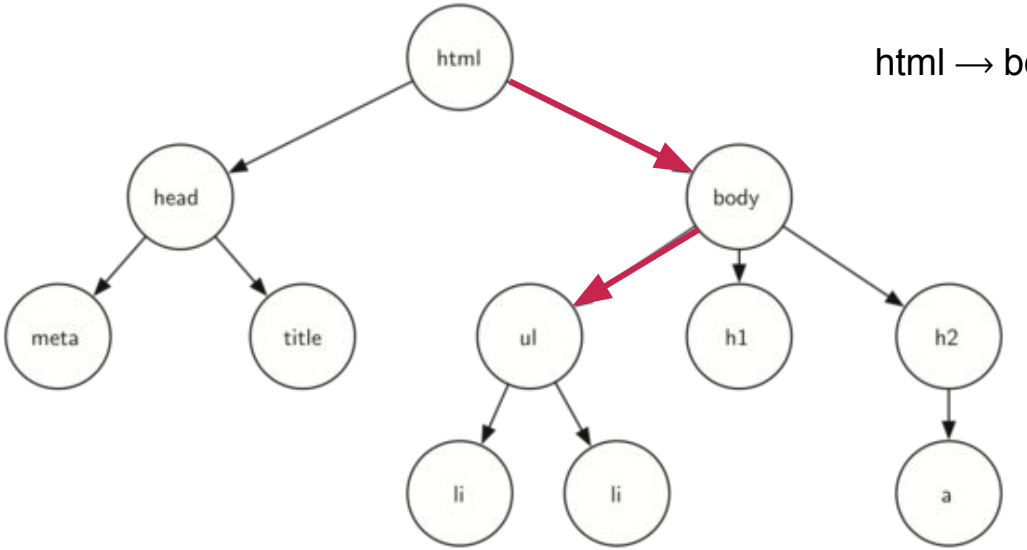
Vocabulário

- Nó
- Aresta
- **Raiz**
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



Vocabulário

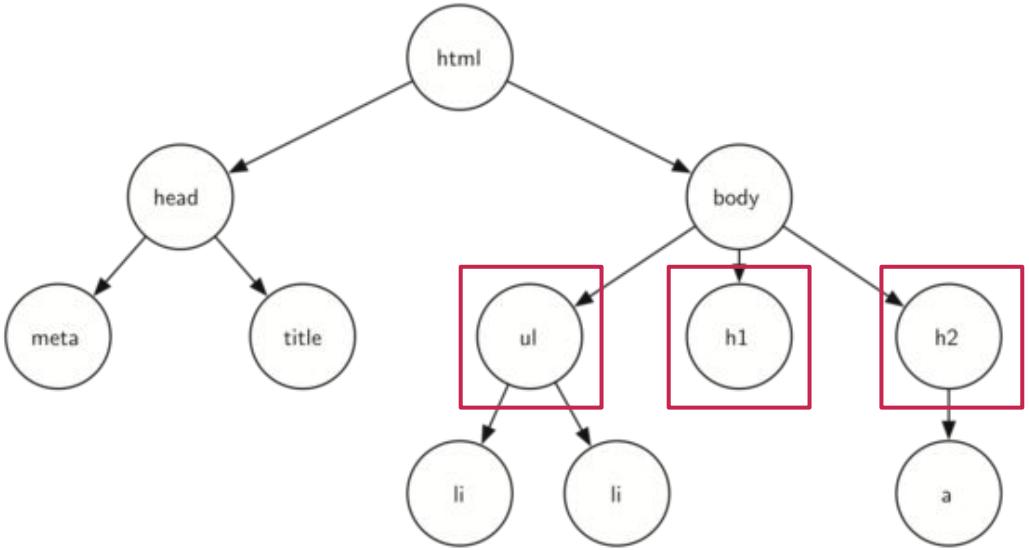
- Nó
- Aresta
- Raiz
- **Caminho**
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



html ⟶ body ⟶ ul

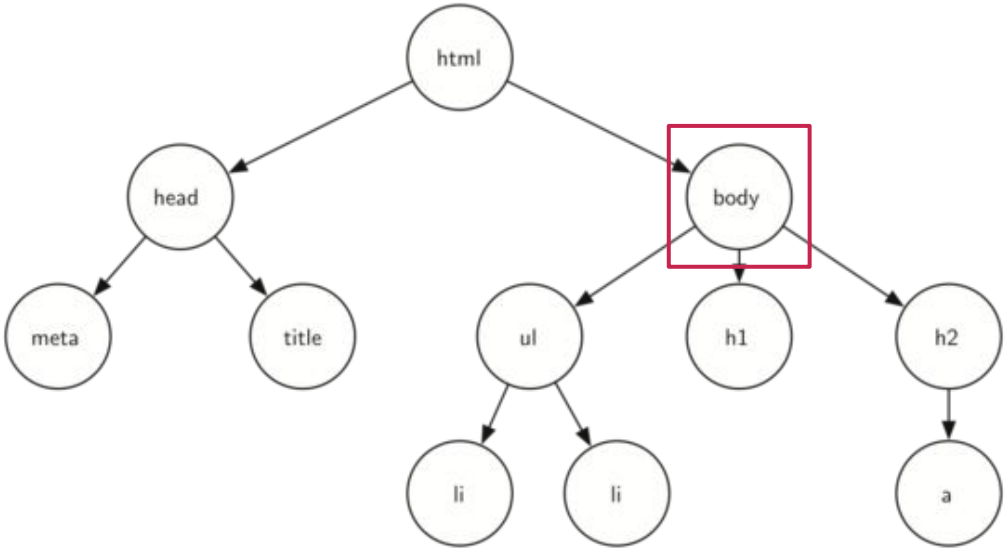
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- **Filho**
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



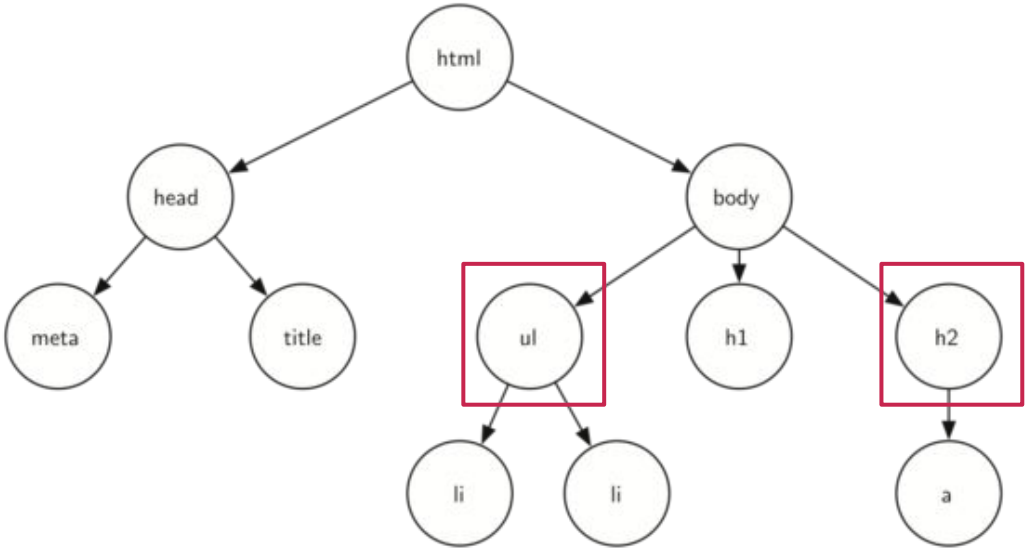
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- **Pai**
- Irmãos
- Subárvore
- Nó folha
- Nível
- Altura



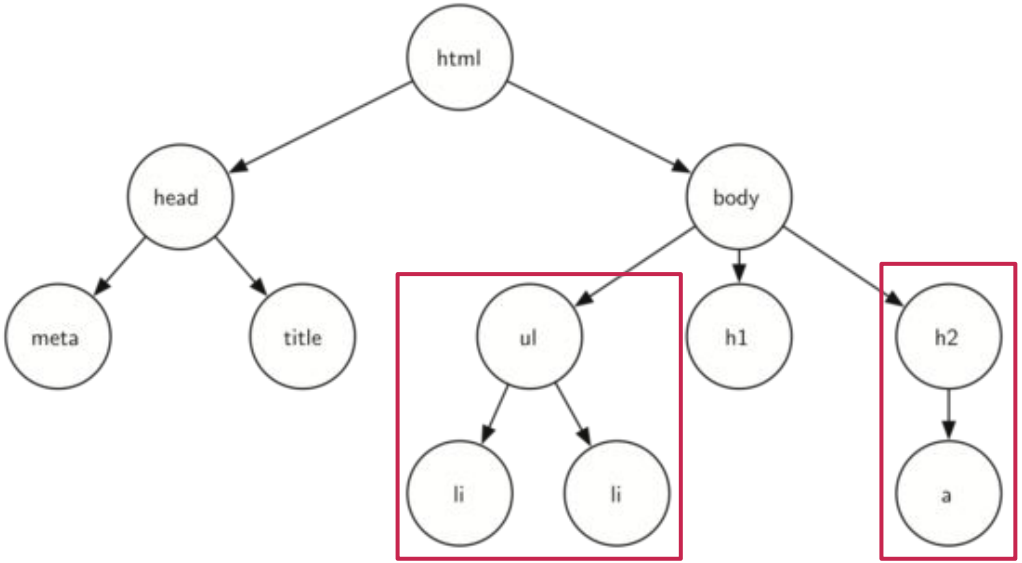
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- **Irmãos**
- Subárvore
- Nó folha
- Nível
- Altura



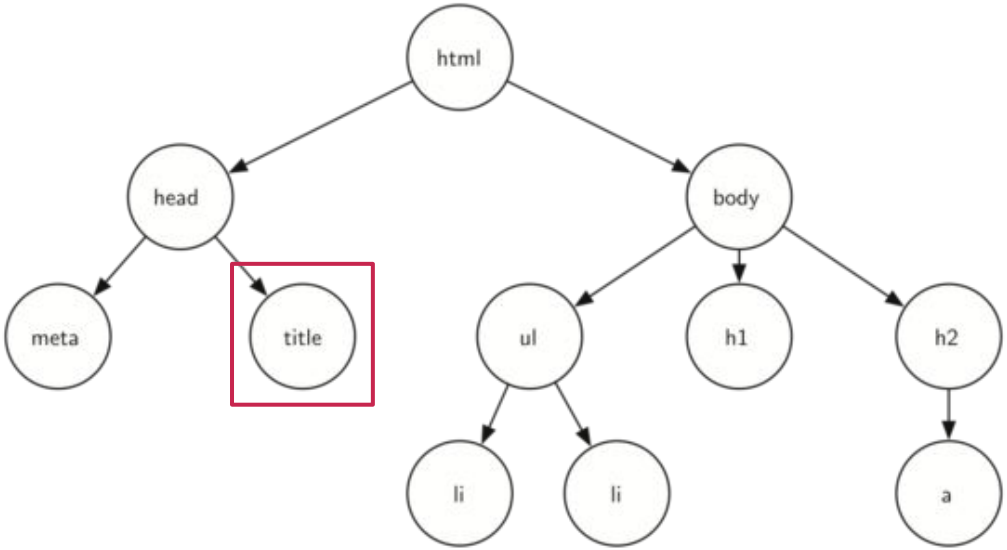
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- **Subárvore**
- Nó folha
- Nível
- Altura



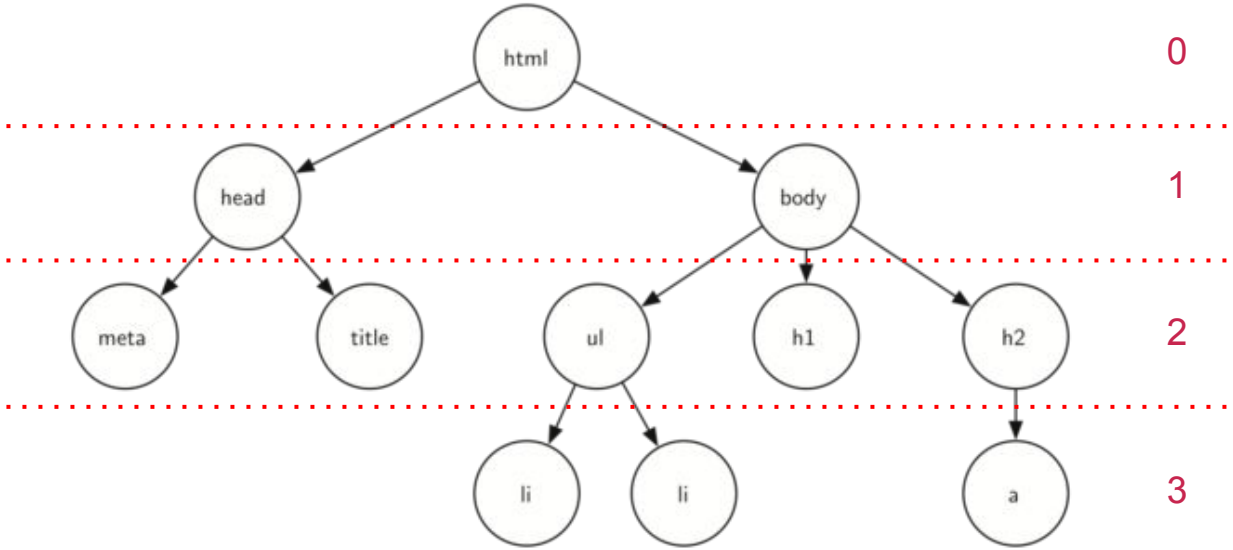
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- **Nó folha**
- Nível
- Altura



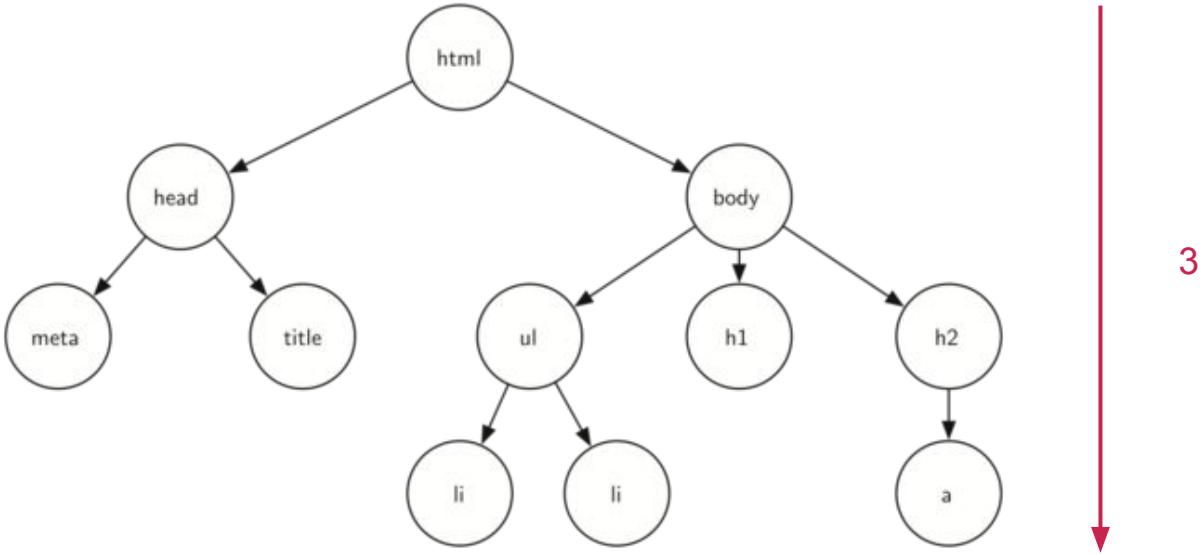
Vocabulário

- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- **Nível**
- Altura

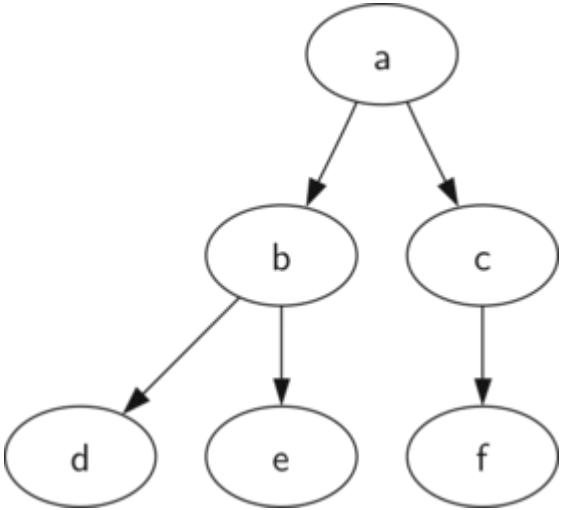


Vocabulário

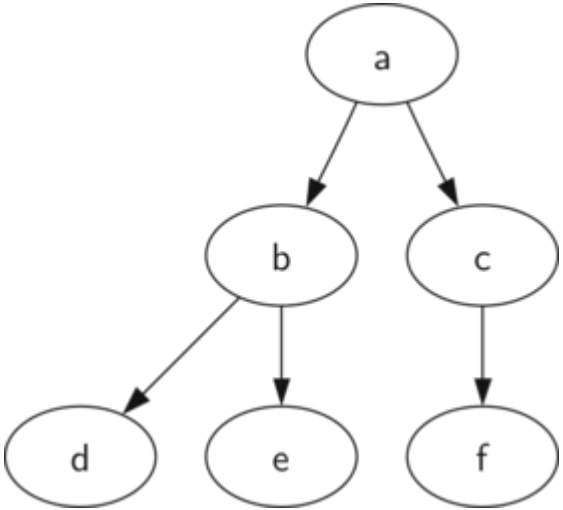
- Nó
- Aresta
- Raiz
- Caminho
- Filho
- Pai
- Irmãos
- Subárvore
- Nó folha
- Nível
- **Altura**



Representação I - Listas



Representação I - Listas




```
myTree = ['a',   #root
            ['b',  #left subtree
                ['d', [], []],
                ['e', [], [] ],
            ['c',  #right subtree
                ['f', [], []],
                [] ]
        ]
```

Representação I - Listas

```
def BinaryTree(r):
    return [r, [], []]
```

Representação I - Listas

```
def BinaryTree(r):
    return [r, [], []]

def insertLeft(root, newBranch):
    t = root.pop(1)
    if len(t) > 1:
        root.insert(1, [newBranch, t, []])
    else:
        root.insert(1, [newBranch, [], []])
    return root
```

Representação I - Listas

```
def BinaryTree(r):
    return [r, [], []]

def insertLeft(root, newBranch):
    t = root.pop(1)
    if len(t) > 1:
        root.insert(1, [newBranch, t, []])
    else:
        root.insert(1, [newBranch, [], []])
    return root

def insertRight(root, newBranch):
    t = root.pop(2)
    if len(t) > 1:
        root.insert(2, [newBranch, [], t])
    else:
        root.insert(2, [newBranch, [], []])
    return root
```

Representação I - Listas

```
def BinaryTree(r):
    return [r, [], []]

def insertLeft(root, newBranch):
    t = root.pop(1)
    if len(t) > 1:
        root.insert(1, [newBranch, t, []])
    else:
        root.insert(1, [newBranch, [], []])
    return root

def insertRight(root, newBranch):
    t = root.pop(2)
    if len(t) > 1:
        root.insert(2, [newBranch, [], t])
    else:
        root.insert(2, [newBranch, [], []])
    return root
```

```
def getRootVal(root):
    return root[0]

def setRootVal(root,newVal):
    root[0] = newVal

def getLeftChild(root):
    return root[1]

def getRightChild(root):
    return root[2]
```

Representação I - Listas

```
def BinaryTree(r):
    return [r, [], []]

def insertLeft(root, newBranch):
    t = root.pop(1)
    if len(t) > 1:
        root.insert(1, [newBranch, t, []])
    else:
        root.insert(1, [newBranch, [], []])
    return root

def insertRight(root, newBranch):
    t = root.pop(2)
    if len(t) > 1:
        root.insert(2, [newBranch, [], t])
    else:
        root.insert(2, [newBranch, [], []])
    return root
```

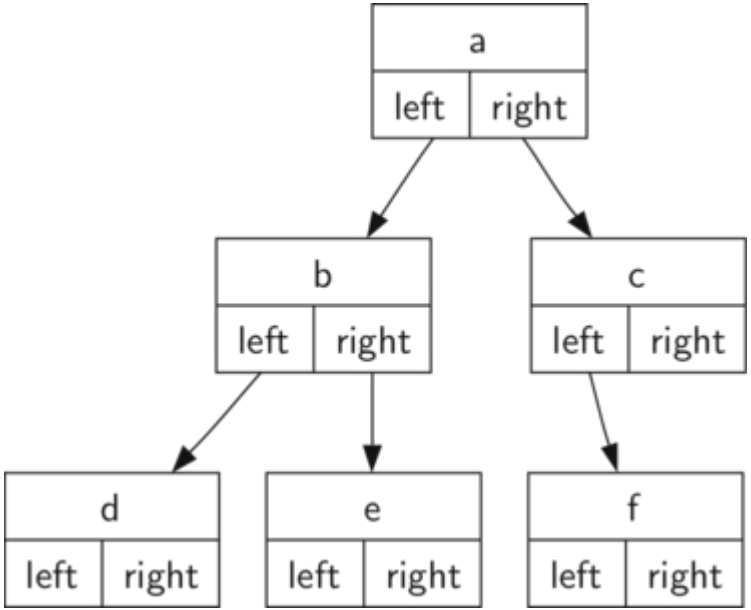
```
32 r = BinaryTree(3)
33 insertLeft(r,4)
34 insertLeft(r,5)
35 insertRight(r,6)
36 print(r)
37
```

```
[3, [5, [4, [], []], []], [6, [], []]]
```

```
32 r = BinaryTree(3)
33 insertLeft(r,4)
34 insertLeft(r,5)
35 insertRight(r,6)
36 insertRight(r,7)
37 print(r)
```

```
[3, [5, [4, [], []], []], [7, [], [6, [], []]]]
```

Representação II - Objetos



Representação II - Objetos

```
class BinaryTree:

    def __init__(self,rootObj):
        self.key = rootObj
        self.leftChild = None
        self.rightChild = None
```

Representação II - Objetos

```
class BinaryTree:

    def __init__(self,rootObj):
        self.key = rootObj
        self.leftChild = None
        self.rightChild = None

    def insertLeft(self,newNode):
        if self.leftChild == None:
            self.leftChild = BinaryTree(newNode)
        else:
            t = BinaryTree(newNode)
            t.leftChild = self.leftChild
            self.leftChild = t
```

Representação II - Objetos

```
class BinaryTree:

    def __init__(self,rootObj):
        self.key = rootObj
        self.leftChild = None
        self.rightChild = None

    def insertLeft(self,newNode):
        if self.leftChild == None:
            self.leftChild = BinaryTree(newNode)
        else:
            t = BinaryTree(newNode)
            t.leftChild = self.leftChild
            self.leftChild = t
```

```
def insertRight(self,newNode):
    if self.rightChild == None:
        self.rightChild = BinaryTree(newNode)
    else:
        t = BinaryTree(newNode)
        t.rightChild = self.rightChild
        self.rightChild = t
```

Representação II - Objetos

```
class BinaryTree:

    def __init__(self,rootObj):
        self.key = rootObj
        self.leftChild = None
        self.rightChild = None

    def insertLeft(self,newNode):
        if self.leftChild == None:
            self.leftChild = BinaryTree(newNode)
        else:
            t = BinaryTree(newNode)
            t.leftChild = self.leftChild
            self.leftChild = t
```

```
    def insertRight(self,newNode):
        if self.rightChild == None:
            self.rightChild = BinaryTree(newNode)
        else:
            t = BinaryTree(newNode)
            t.rightChild = self.rightChild
            self.rightChild = t

    def getRightChild(self):
        return self.rightChild

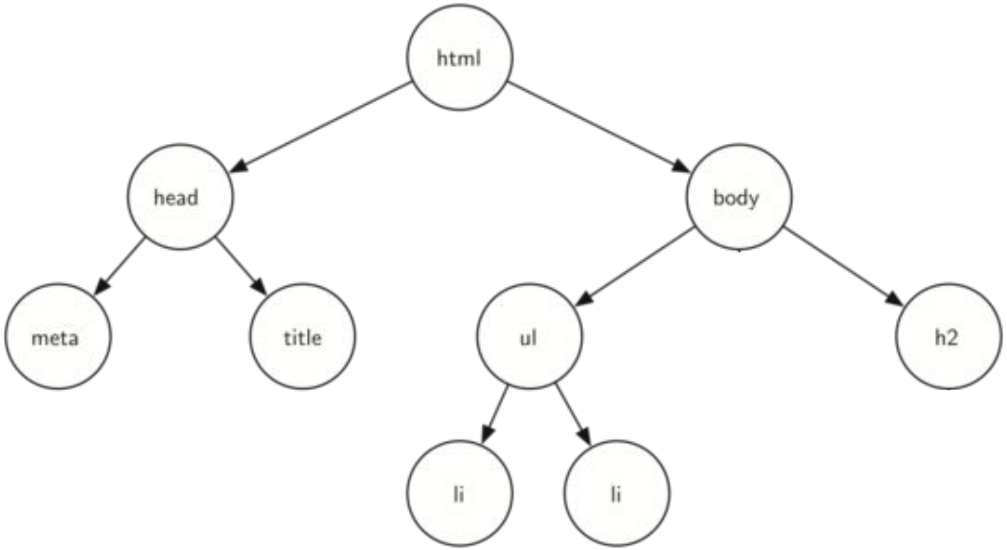
    def getLeftChild(self):
        return self.leftChild

    def setRootVal(self,obj):
        self.key = obj

    def getRootVal(self):
        return self.key
```

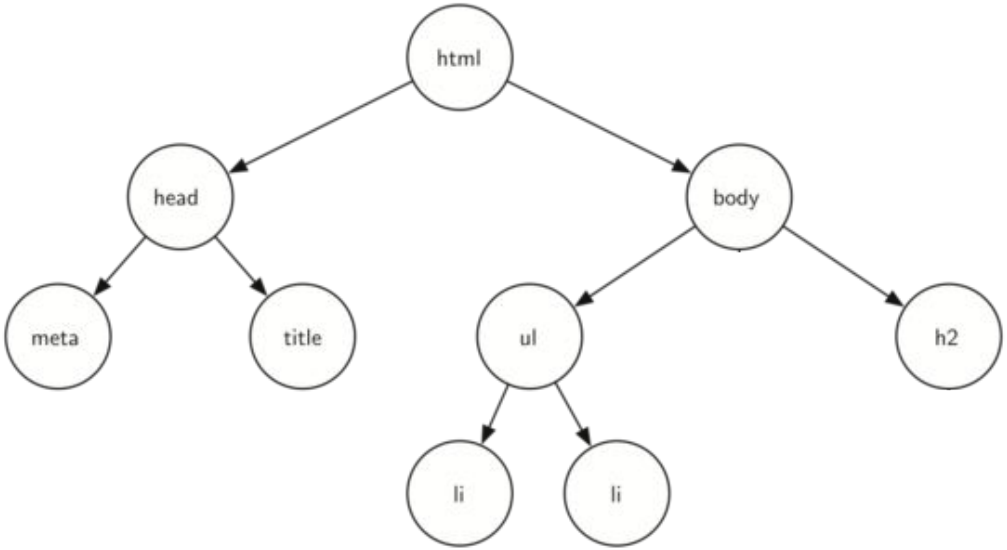
Travessias

- Preorder
- Inorder
- Postorder



Travessias

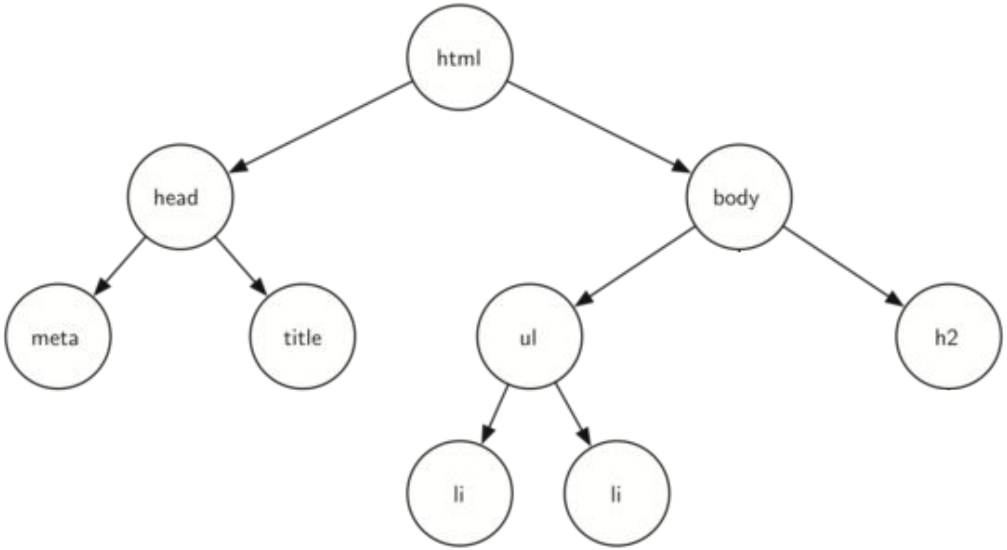
- **Preorder**
- Inorder
- Postorder



html ⟶ head ⟶ meta ⟶ title ⟶ body ⟶ ul ⟶ li ⟶ li ⟶ h2

Travessias

- **Preorder**
- Inorder
- Postorder

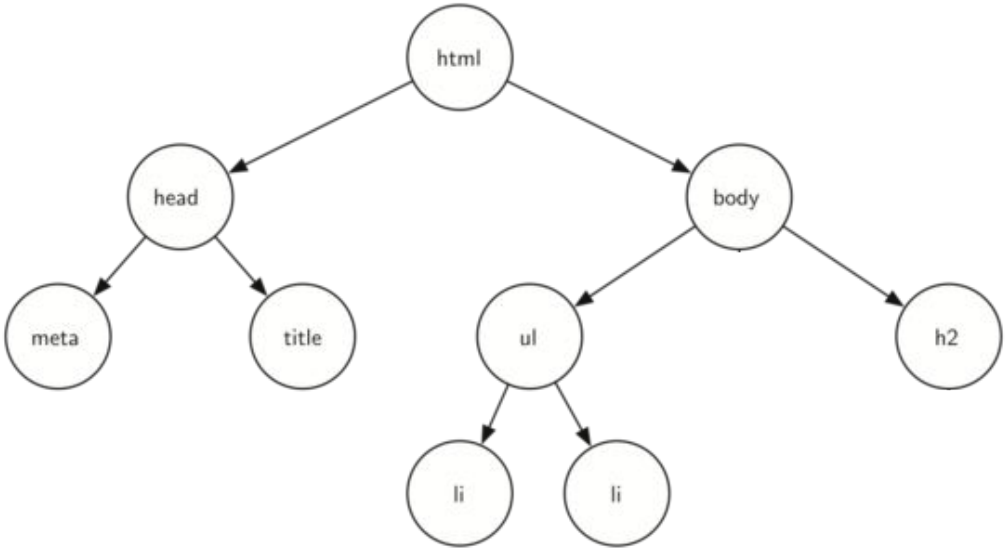


html ⟶ head ⟶ meta ⟶ title ⟶ body ⟶ ul ⟶ li ⟶ li ⟶ h2

```
def preorder(tree):
    if tree:
        print(tree.getRootVal())  # processa o nodo
        preorder(tree.getLeftChild())
        preorder(tree.getRightChild())
```

Travessias

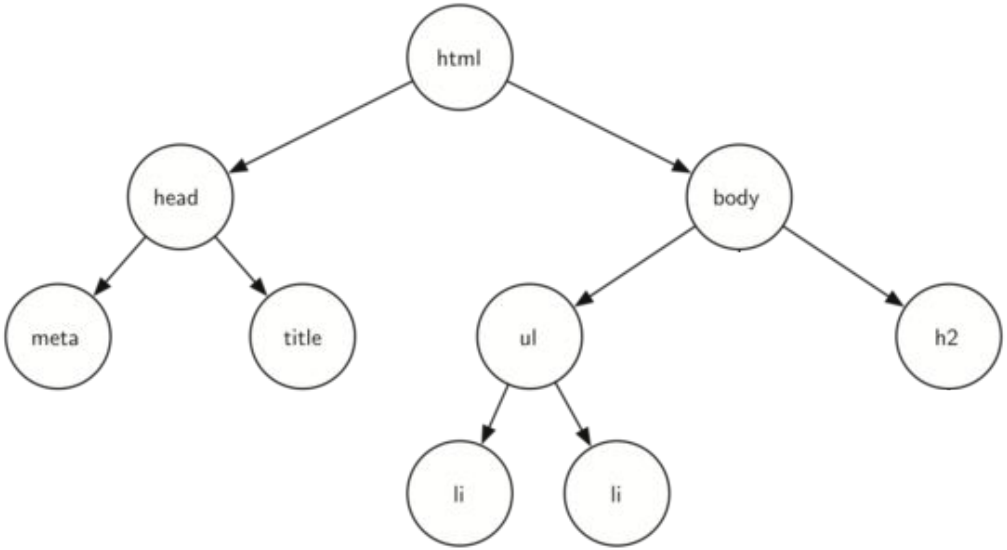
- Preorder
- **Inorder**
- Postorder



meta ⟶ head ⟶ title ⟶ html ⟶ li ⟶ ul ⟶ li ⟶ body ⟶ h2

Travessias

- Preorder
- **Inorder**
- Postorder

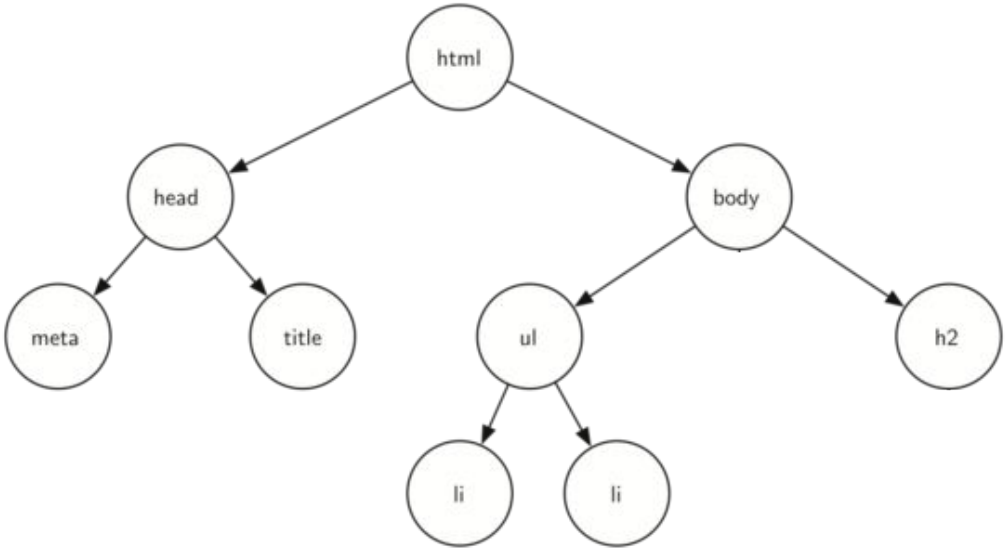


meta ⟶ head ⟶ title ⟶ html ⟶ li ⟶ ul ⟶ li ⟶ body ⟶ h2

```
def inorder(tree):
    if tree:
        inorder(tree.getLeftChild())
        print(tree.getRootVal())  # processa o nodo
        inorder(tree.getRightChild())
```

Travessias

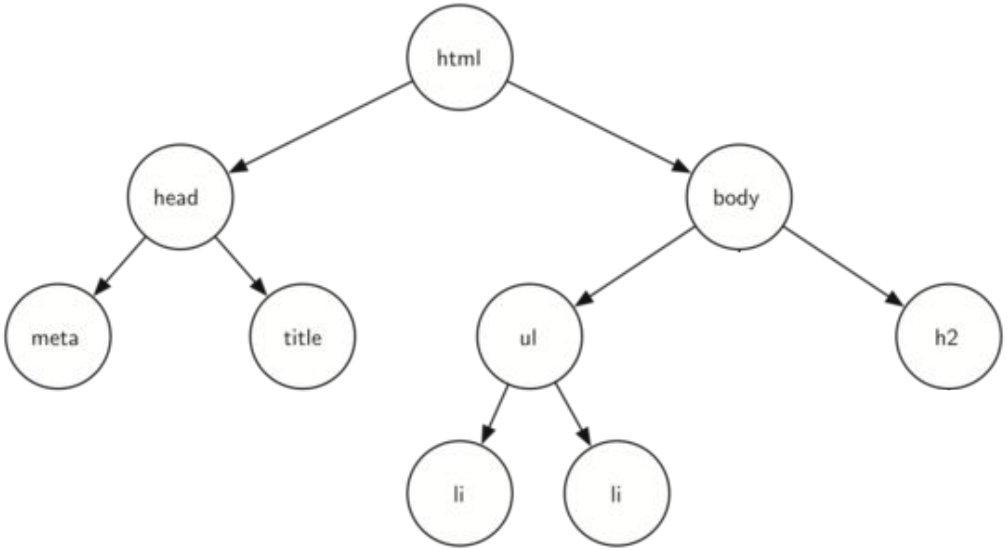
- Preorder
- Inorder
- **Postorder**



meta ⟶ title ⟶ head ⟶ li ⟶ li ⟶ ul ⟶ h2 ⟶ body ⟶ html

Travessias

- Preorder
- Inorder
- **Postorder**



meta → title → head → li → li → ul → h2 → body → html

```
def postorder(tree):
    if tree:
        postorder(tree.getLeftChild())
        postorder(tree.getRightChild())
        print(tree.getRootVal())  # processa o nodo
```

Resumo

- Árvore é uma estrutura hierárquica de natureza recursiva.
- Podemos implementar árvores usando listas.
- Podemos implementar árvores usando classes e referências.
- A travessia em uma árvore pode ser feita de diferentes formas.